

Recap

Creating/naming objects

Creating/naming objects

Objects can be created with either the `<-` or the `=` command

```
1 x1 = 3 # name = object
2 x1
```

```
[1] 3
```

```
1 x2 <- 3 # name <- object
2 x2
```

```
[1] 3
```

```
1 x1 == x2 # are the two objects identical?
```

```
[1] TRUE
```

Object names must start with a letter!! NOT “name” = ...but name = ...

Logical Operators

In R we can combine several relations to evaluate a given statement.

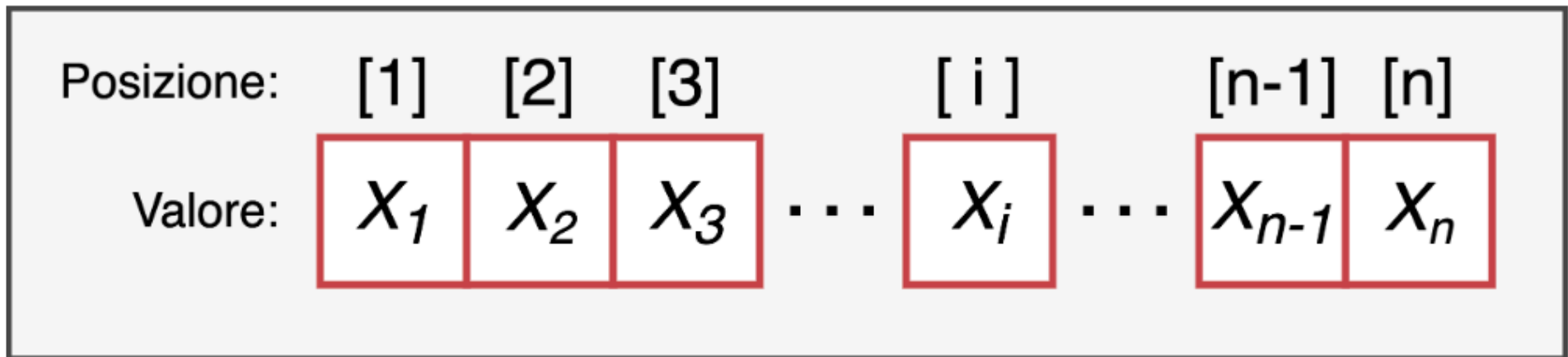
```
1 x = 30 # Assign the value 30 to x.
```

Function	Name	Example	Result
&	Conjunction (AND)	x>25 & x<60	TRUE
	Inclusive disjunction (OR)	x>25 x>60	TRUE
!	Negation (NOT)	!(x<18)	TRUE

Vectors

Vectors

- **a value:** the value of the element, which can be of any type, for example a number or a string of characters
- **a position index:** a positive integer that identifies its position within the vector.



Creating a vector

Vectors can be created with the `c()` command, writing inside the brackets the values of the elements in the desired order, separated by commas.

```
1 num_vect = c(1,2,3,4)
2 # you can also use the seq() function
3 num_vect_seq = seq(from = 1,to = 4, by = 1)
4
5 num_vect;num_vect_seq
```

```
[1] 1 2 3 4
```

```
[1] 1 2 3 4
```

```
1 char_vect = c("R","R","R","ok")
2 # you can also use the rep() function
3 char_vect_rep = c(rep("R", 3), "ok")
4
5 char_vect;char_vect_rep
```

```
[1] "R"  "R"  "R"  "ok"
```

```
[1] "R"  "R"  "R"  "ok"
```

Vector type

A vector must be made of **elements all of the same type!**

```
1 wrong = c(1,2,3,"don't know", 4)
2 class(wrong)
```

```
[1] "character"
```

```
1 wrong
```

```
[1] "1"          "2"          "3"          "don't know" "4"
```

Otherwise you “risk” everything being converted to character.

```
1 correct = c(1,2,3,NA, 4)
2 class(correct)
```

```
[1] "numeric"
```

```
1 correct
```

```
[1] 1 2 3 NA 4
```

is.* & as.*

We can test or convert (when possible) the type of a vector with the `is.*` & `as.*` functions

```
1 is.character(char_vect)
```

```
[1] TRUE
```

```
1 as.numeric(char_vect) #!!
```

```
[1] NA NA NA NA
```

```
1 is.numeric(num_vect)
```

```
[1] TRUE
```

```
1 as.character(num_vect) #!!
```

```
[1] "1" "2" "3" "4"
```

```
1 logi_vect = c(TRUE,FALSE,TRUE)
2 is.logical(logi_vect)
```

```
[1] TRUE
```

```
1 as.numeric(logi_vect)
```

```
[1] 1 0 1
```

Indexing

We can select, remove and extract elements simply by using the position index inside square brackets `vector[pos]`

```
1 # Create a vector of 20 random numbers drawn between 1 and 100
2 my_vect = round(runif(n = 20,min = 1,max = 100))
3 my_vect
```

```
[1] 20 90 25 46 16 59 50  3 45 66 83 16 65 22 71  9 82  8 92 54
```

```
1 my_vect[1] # extract the first element
```

```
[1] 20
```

```
1 my_vect[1:5] # extract the first 5 elements
```

```
[1] 20 90 25 46 16
```

```
1 my_vect[c(1,4,2,9)] # extract elements of our choice
```

```
[1] 20 46 90 45
```

```
1 my_vect[length(my_vect)] # last element (why?)
```

```
[1] 54
```

Negative indexing

In the same way, we can extract all the elements of the vector except some

```
1 my_vect[-c(1)] # all except the first element
```

```
[1] 90 25 46 16 59 50  3 45 66 83 16 65 22 71  9 82  8 92 54
```

```
1 my_vect[-c(1:10)] # all except the first 10
```

```
[1] 83 16 65 22 71  9 82  8 92 54
```

Logical Indexing

Indexing by position is the simplest and most intuitive approach. We can also select using **TRUE** and **FALSE** values: we can extract elements from the vector based on specific logical conditions

```
1 numbers = 1:7  
2 numbers
```

```
[1] 1 2 3 4 5 6 7
```

```
1 numbers[numbers<2]
```

```
[1] 1
```

Factors

Factors

Factors can be created either by converting a character vector with the **as.factor()** command or by explicitly creating a factor with the **factor()** command

```
1 char_vect = rep(c("hello","ciao", "hola"), each = 2)
2 my_fact = as.factor(char_vect)
3
4 my_fact2 = factor(rep(c("hello","ciao", "hola"), each = 2))
5
6 my_fact == my_fact2
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE
```

Properties of factors

In practice, they assign a label to an integer value:

```
1 typeof(my_fact)
```

```
[1] "integer"
```

```
1 as.integer(my_fact)
```

```
[1] 2 2 1 1 3 3
```

```
1 my_fact
```

```
[1] hello hello ciao ciao hola hola
```

```
Levels: ciao hello hola
```

Factor levels

Factors store `levels()` as metadata, regardless of which ones are actually present in the vector

```
1 levels(my_fact)
```

```
[1] "ciao" "hello" "hola"
```

```
1 my_fact2 = my_fact[my_fact!="ciao"]
2 my_fact2
```

```
[1] hello hello hola hola
Levels: ciao hello hola
```

However, we can drop the levels that are no longer needed with the `droplevels()` command

```
1 droplevels(my_fact2)
```

```
[1] hello hello hola hola
Levels: hello hola
```

We can also rename the levels of a factor

```
1 # the factor I created
2 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

```
1 # create another factor "my_fact_lev" identical to "my_fact"
2 my_fact_lev = my_fact
3 my_fact_lev == my_fact
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE
```

```
1 # rename the levels of the factor
2 levels(my_fact_lev) = c("italian","english","spanish")
3
4 my_fact_lev
```

```
[1] english english italian italian spanish spanish
Levels: italian english spanish
```

```
1 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola
Levels: ciao hello hola
```

We can both reorder and rename the levels of a factor with the `factor()` function

```
1 my_fact
```

```
[1] hello hello ciao  ciao  hola  hola  
Levels: ciao hello hola
```

```
1 # by default levels are assigned in alphabetical order  
2 as.integer(my_fact)
```

```
[1] 2 2 1 1 3 3
```

```
1 new_fact = factor(my_fact, levels = c("hello","ciao","hola"))  
2 new_fact
```

```
[1] hello hello ciao  ciao  hola  hola  
Levels: hello ciao hola
```

```
1 as.integer(new_fact)
```

```
[1] 1 1 2 2 3 3
```

```
1 new_fact2 = factor(my_fact, levels = c("hello","ciao","hola"),  
2                      labels = c("english","italian","spanish"))  
3 new_fact2
```

```
[1] english english italian italian spanish spanish  
Levels: english italian spanish
```

Remember: the *levels* argument inside the `factor()` function is used to **reorder** the levels of the factor, while the `levels()` function **renames** them, it does not reorder them!

```
1 # the factor I created
2 my_fact
```

```
[1] hello hello ciao ciao hola hola
Levels: ciao hello hola
```

```
1 as.integer(my_fact)
```

```
[1] 2 2 1 1 3 3
```

```
1 levels(my_fact) = c("hello","ciao","hola")
2
3 # What do you notice compared to the my_fact vector shown above?
4 my_fact
```

```
[1] ciao ciao hello hello hola hola
Levels: hello ciao hola
```

```
1 as.integer(my_fact)
```

```
[1] 2 2 1 1 3 3
```

Lists

Lists

Lists are flexible structures that can contain objects of different types and different sizes (each element can itself be a list)

```
1 my_list = list("hello!", 1:3, c(TRUE,FALSE),  
2               list("hello!", 1:3, c(TRUE,FALSE)))  
3 str(my_list)
```

List of 4

```
$ : chr "hello!"  
$ : int [1:3] 1 2 3  
$ : logi [1:2] TRUE FALSE  
$ :List of 3  
..$ : chr "hello!"  
..$ : int [1:3] 1 2 3  
..$ : logi [1:2] TRUE FALSE
```

Lists

Although a list is one-dimensional, it develops in depth

```
1 str(my_list)
```

List of 4

```
$ : chr "hello!"
```

```
$ : int [1:3] 1 2 3
```

```
$ : logi [1:2] TRUE FALSE
```

```
$ :List of 3
```

```
..$ : chr "hello!"
```

```
..$ : int [1:3] 1 2 3
```

```
..$ : logi [1:2] TRUE FALSE
```

Lists

First Level: list

```
1 str(my_list[2])
```

List of 1

```
$ : int [1:3] 1 2 3
```

Second Level: numeric vector

```
1 str(my_list[[2]])
```

```
int [1:3] 1 2 3
```

Third Level: integer

```
1 str(my_list[[2]][1]) # integer
```

```
int 1
```

Lists

We can assign names to the elements of a list...

```
1 my_list_nam = list(word = "hello!", numbers = 1:3,  
2                   logical = c(TRUE, FALSE),  
3                   list = list("hello!", 1:3, c(TRUE, FALSE)))  
4 my_list_nam
```

```
$word  
[1] "hello!"  
  
$numbers  
[1] 1 2 3  
  
$logical  
[1] TRUE FALSE
```

```
$list  
$list[[1]]  
[1] "hello!"
```

```
$list[[2]]  
[1] 1 2 3
```

```
$list[[3]]  
[1] TRUE FALSE
```

Lists

... and access them by calling them by “name” with the **\$** operator

```
1 my_list_nam$word
```

```
[1] "hello!"
```

```
1 my_list_nam$list # list created as an element of the list
```

```
[[1]]
```

```
[1] "hello!"
```

```
[[2]]
```

```
[1] 1 2 3
```

```
[[3]]
```

```
[1] TRUE FALSE
```

Lists - Attributes

Like vectors, lists also have a length (**length()**) and possibly names (**names()**). The **str()** (structure) command is very useful for lists because it gives an overview of their structure:

```
1 length(my_list_nam); names(my_list_nam)
```

```
[1] 4
```

```
[1] "word"      "numbers" "logical" "list"
```

```
1 str(my_list_nam)
```

```
List of 4
```

```
$ word : chr "hello!"
```

```
$ numbers: int [1:3] 1 2 3
```

```
$ logical: logi [1:2] TRUE FALSE
```

```
$ list :List of 3
```

```
..$ : chr "hello!"
```

```
..$ : int [1:3] 1 2 3
```

```
..$ : logi [1:2] TRUE FALSE
```

Lists - Indexing

The difference between the square brackets is whether we want to subset the list, getting another list, or directly access the inner element:

```
1 my_list_nam[1] # getting another list
```

```
$word  
[1] "hello!"
```

```
1 my_list_nam[[1]] # accessing the inner element
```

```
[1] "hello!"
```

Lists - Indexing

If we want to select several elements (i.e. actually subset the list) we must always use single square brackets:

```
1 # list made of the first two elements of my_list_nam
2 my_list_nam[1:2]
```

\$word

```
[1] "hello!"
```

\$numbers

```
[1] 1 2 3
```

Lists - Nested Indexing

Lists have a one-dimensional structure that can develop in depth. To select nested elements we can chain several brackets:

```
1 my_list_nam[[4]]
```

```
[[1]]  
[1] "hello!"
```

```
[[2]]  
[1] 1 2 3
```

```
[[3]]  
[1] TRUE FALSE
```

```
1 my_list_nam[[4]][[1]]
```

```
[1] "hello!"
```

```
1 my_list_nam[[4]][[2]]
```

```
[1] 1 2 3
```

Matrices

Matrices

Matrices are a **two-dimensional** data structure (characterized by 2 dimensions, `dim()`), where the number of rows is dimension 1 and the number of columns is dimension 2.

```
1 my_mat = matrix(data = 1:10, nrow = 2, ncol = 5, byrow = FALSE)
2 my_mat
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	3	5	7	9
[2,]	2	4	6	8	10

```
1 matrix(data = 1:10, nrow = 2, ncol = 5, byrow = TRUE)
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	1	2	3	4	5
[2,]	6	7	8	9	10

Matrices - Properties

- They can contain **only one type** of data
- Since they are **two-dimensional**, we need two position indices (rows and columns) to identify an element
- They can be seen as a **set** of single **vectors**

Matrices - Properties

The number of rows and columns does not have to be the same (square matrix), but the size of the matrix must be compatible with the data vector:

```
1 matrix(data = 1:10, ncol = 3, nrow = 3)
```

Warning in matrix(data = 1:10, ncol = 3, nrow = 3): data length [10] is not a sub-multiple or multiple of the number of rows [3]

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

Matrices

What does R do by default?

```
1 matrix(data = 1:10, ncol = 3, nrow = 3)
```

	[,1]	[,2]	[,3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

```
1 matrix(data = 1:2, ncol = 3, nrow = 3)
```

	[,1]	[,2]	[,3]
[1,]	1	2	1
[2,]	2	1	2
[3,]	1	2	1

warnings: the function informs us about something potentially problematic, but (more or less!!) everything is fine

Matrices - Indexing

To identify one or more elements of a matrix we need row and/or column indices separated by a comma, again inside square brackets: `matrix[row, column]`

```
1 my_mat
```

```
      [,1] [,2] [,3] [,4] [,5]  
[1,]     1     3     5     7     9  
[2,]     2     4     6     8    10
```

```
1 my_mat[1,1]
```

```
[1] 1
```

Matrices - Indexing

We can also select an entire row or column

```
1 my_mat[1,]
```

```
[1] 1 3 5 7 9
```

```
1 my_mat[:,1]
```

```
[1] 1 2
```

Matrices - Indexing

As with vectors, here too we can apply logical indexing:

```
1 my_mat>2
```

```
      [,1] [,2] [,3] [,4] [,5]  
[1,] FALSE TRUE TRUE TRUE TRUE  
[2,] FALSE TRUE TRUE TRUE TRUE
```

```
1 # All the elements greater than two  
2 my_mat[my_mat>2]
```

```
[1] 3 4 5 6 7 8 9 10
```

Vectors and Matrices

Vectors are created with the `c()` function and can be concatenated using the same function:

```
1 my_vect1 = c(1:4); my_vect2 = c(5:10)
2 my_vect12 = c(my_vect1,my_vect2)
3 my_vect12
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Matrices

Similarly, matrices can be joined together with the `cbind()` and `rbind()` commands:

```
1 my_mat = matrix(1:4,2,2)
2
3 cbind(my_mat, my_mat)
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	3	1	3
[2,]	2	4	2	4

```
1 rbind(my_mat, my_mat)
```

	[,1]	[,2]
[1,]	1	3
[2,]	2	4
[3,]	1	3
[4,]	2	4

Dataframe

Dataframe

The dataframe is the most “complex”, useful and powerful structure in R. Intuitively it is an excel sheet, while from R’s point of view it is a type of list with some properties/restrictions

- each element of the list is a **vector** with an **associated name** (aka a column)
- each list/column must have the same number of elements
- as a consequence, each row has the same number of elements (**rectangular structure**)

Creating a dataframe

With the `data.frame()` function we can create a dataframe...

```
1 # Create a dataframe with 3 columns
2 my_df = data.frame( col1 = 1:4, col2 = letters[1:4],
3                     col3 = rnorm(4))
4 my_df
```

	col1	col2	col3
1	1	a	0.9518920
2	2	b	-1.9647190
3	3	c	-0.1013861
4	4	d	-0.7723165

Attributes

The **dataframe** has both the attributes of a **list**, i.e. the ***names***, and the attributes of a **matrix**, i.e. the ***dimensions*** (rows and columns)

```
1 typeof(my_df)
```

```
[1] "list"
```

```
1 attributes(my_df)
```

```
$names
```

```
[1] "col1" "col2" "col3"
```

```
$class
```

```
[1] "data.frame"
```

```
$row.names
```

```
[1] 1 2 3 4
```

```
1 dim(my_df)
```

```
[1] 4 3
```

Indexing

We can use either square brackets `[]` or the dollar sign `$`

```
1 my_df[1] # extract a 4x1 data.frame
```

```
  col1  
1     1  
2     2  
3     3  
4     4
```

```
1 my_df[[1]]# extract the first column of the data.frame
```

```
[1] 1 2 3 4
```

```
1 my_df$col1# extract the first column of the data.frame
```

```
[1] 1 2 3 4
```

```
1 my_df[1,1]# extract the first element of the first column of the data.frame
```

```
[1] 1
```

Indexing - Relational operators

One of the most common operations you will deal with is extracting/evaluating a subset of the values in your dataset:

```
1 # keep only the rows where the values of column1 are greater than 2
2 my_df[my_df$col1 > 2, ]
```

	col1	col2	col3
3	3	c	-0.1013861
4	4	d	-0.7723165

```
1 my_df[my_df[1] > 2, ]
```

	col1	col2	col3
3	3	c	-0.1013861
4	4	d	-0.7723165

Examples

	col1	col2	col3
1	1	a	0.9518920
2	2	b	-1.9647190
3	3	c	-0.1013861
4	4	d	-0.7723165

- `my_df[my_df$col1 > 1 & my_df$col1 < 4, ]`
 - 2 b -0.5630882
 - 3 c -0.5672335
- `my_df[my_df$col1== 1, "col3"]`
 - -1.481937
- `my_df[my_df$col1== 1, 3]`
 - -1.481937

Indexing

There are also alternative, more compact ways of indexing. For example, using the `subset()` function:

```
1 subset(iris, subset = Species == "setosa" & Petal.Length > 1.7)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
25	4.8	3.4	1.9	0.2	setosa
45	5.1	3.8	1.9	0.4	setosa

equivalent to:

```
1 iris[iris$Species == "setosa" & iris$Petal.Length > 1.7,]
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
25	4.8	3.4	1.9	0.2	setosa
45	5.1	3.8	1.9	0.4	setosa

We can also select columns rather than rows with the *select* argument:

```
1 head(subset(iris, select = c(Sepal.Length, Species)), n = 3)
```

	Sepal.Length	Species
1	5.1	setosa
2	4.9	setosa
3	4.7	setosa

We can also combine the two:

```
1 head(subset(iris, Species == "setosa" & Sepal.Length > 4,  
2           c(Sepal.Length, Species)), n = 3)
```

	Sepal.Length	Species
1	5.1	setosa
2	4.9	setosa
3	4.7	setosa

equivalent to:

```
1 head(iris[iris$Species == "setosa" & iris$Sepal.Length > 4,  
2       c("Sepal.Length","Species")],n = 3)
```

	Sepal.Length	Species
1	5.1	setosa
2	4.9	setosa
3	4.7	setosa

\$\$\$\$\$\$

Most of the time you will access variables with the `$` operator. This command can also be used to create a new variable...

```
1 iris$sum = iris$Sepal.Length + iris$Sepal.Width
2 str(iris)
```

```
'data.frame':  150 obs. of  6 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ sum          : num  8.6 7.9 7.9 7.7 8.6 9.3 8 8.4 7.3 8 ...
```

EXAM

<https://etherpad.wikimedia.org/p/arca-corsoR>